

# CREDIT CARD FRAUD DETECTION SYSTEM

A Tri-Model Architecture Leveraging Decision Trees, XGBoost, and Heterogeneous Graph  
Neural Networks for Transaction Security

Presented By:  
M. R Lekshmi Priya (253011)  
Hemalekshmi R (253114)  
Ashna Jabin N.K (253107)



# Introduction & Problem Statement

- **The Problem:** Credit card fraud costs the global economy billions annually.
  - **The Challenge:** Fraudulent patterns are "needles in a haystack" (imbalanced data) and are constantly evolving.
  - **The Goal:** Build a system that not only looks at transaction numbers but also at how users, devices, and cards are interconnected.
- 

# Dataset Overview



- **Source:** IEEE Computational Intelligence Society (IEEE-CIS) dataset from Kaggle.
  - **Complexity:** Contains two main tables: Transactions (amounts, timestamps) and Identity (device info, network details).
  - **Scale:** Over 500,000 transactions with hundreds of features (V-features, C-features, D-features about 870 columns).
  - **Imbalance:** Highlighting that only a tiny percentage of the data is actually fraud.
- 

# EDA

- Dataset Overview:

Total Transactions: 590,540 (Original) | Notebook Sample: 177,162 (30%)

Dimensionality: 434 Initial Features (Transaction + Identity data join).

- Target Class Imbalance - Legitimate: 96.42% | Fraudulent: 3.58%
- Imbalance Ratio: 1 : 26.9 (Requires stratified sampling).
- Missing Value Analysis: Identified 214 columns with >50% missing data (primarily in Vesta and Identity features).
- High sparsity in "V" features suggests the need for dimensionality reduction (PCA).
- Statistical Findings: Transaction amounts follow a long-tail distribution (Outliers identified at \$1,104+).
- Temporal analysis reveals cyclical "midnight" spikes in fraudulent attempts.



# Target Class Distribution



# Feature Engineering & Preprocessing

## 1. Data Cleaning:

Memory Optimization: Downcasted numeric types, reducing RAM usage by ~60%.

Missingness Strategy: Median imputation for numeric gaps; "Unknown" tagging for categorical text.

Outlier Capping: Applied 99th percentile clipping (\$1,104) to stabilize model gradients.

## 2. Feature Transformation & Scaling:

Z-Score Scaling: Normalized all features to Mean=0, Variance=1.

Label Encoding: Transformed 9 categorical features (DeviceType, card4, etc.) into numeric inputs.

## 3. Dimensionality Reduction (PCA):

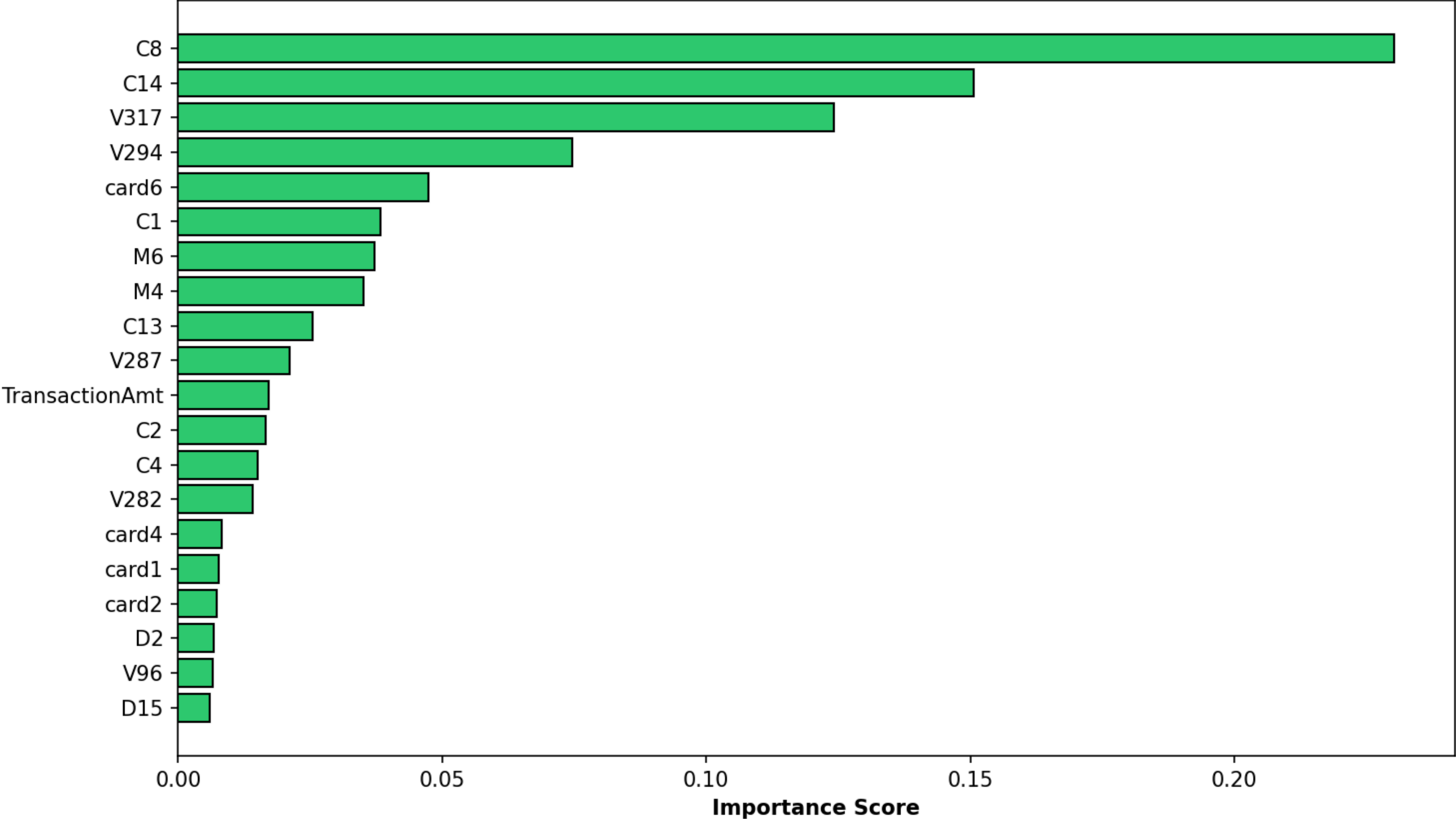
Condensed 339 "V-features" into 50 Principal Components (PCA applied only for XGBoost model)

Maintains 90%+ variance while removing noise and multi-collinearity.

# Decision Tree (Baseline Model)

- Model Objectives:
  - Establish a "glass-box" baseline to understand feature importance.
  - Implement SMOTE (Synthetic Minority Over-sampling Technique) to address the 1:27 class imbalance.
- Training Configuration:
  - Algorithm: Decision Tree Classifier (max\_depth=12).
  - Resampling: Increased fraud representation from 4,439 to 59,786 samples using synthetic generation.
  - Weighting: Utilized class\_weight='balanced' to further penalize misclassified fraud.
- Baseline Results (Test Set):
  - Fraud Detection Rate (Recall): 54.57% (519/951 frauds detected).
  - Precision: 29.29% (Low precision due to aggressive flagging).
  - ROC-AUC: 0.8390.
- The model successfully identified over half of the fraud cases but produced 1,253 false alarms.
- Top Predictors: Features related to Transaction Amount and Card Identity proved most influential.
- Conclusion: Serves as a benchmark; more advanced models (XGBoost/HGNN) are required to reduce "Customer Friction" (False Positives).

**Decision Tree - Top 20 Most Important Features**

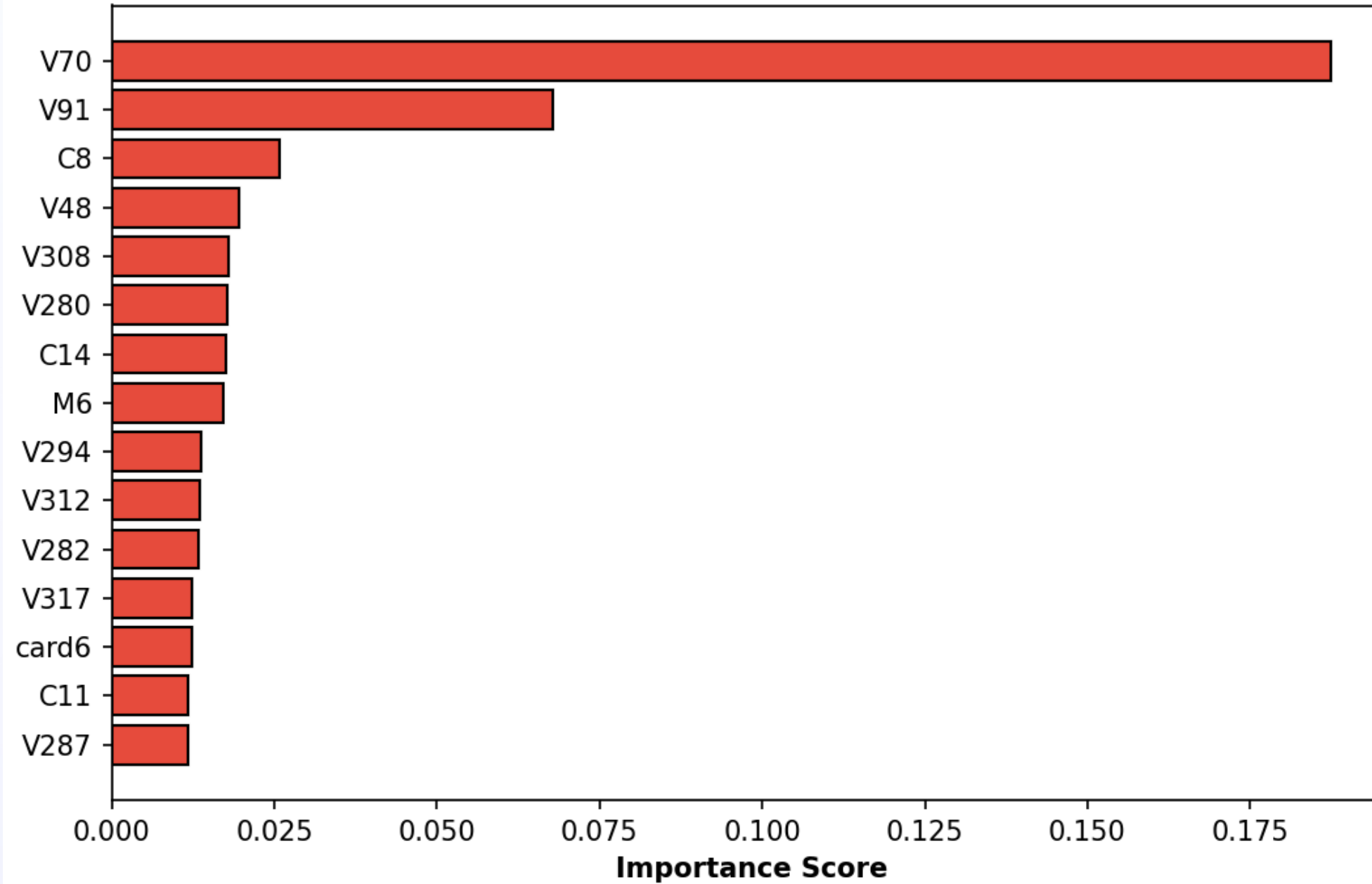




# XGBoost Model

- Algorithm: XGBoost (Extreme Gradient Boosting) using 500 trees and a max depth of 8.
- Imbalance Handling: SMOTE Resampling - Generated 55,347 synthetic samples to balance the training set.
- Cost-Sensitive Learning: Set `scale_pos_weight` = 26.9 to penalize missed fraud more heavily.
- Performance Metrics:
  - ROC-AUC: 0.9198 (Strong overall discriminative power).
  - Recall: 72.66% (Catches nearly 3/4 of all fraud cases).
  - Precision: 31.11% (Trade-off: higher false-alarm rate to ensure security).
- Top 15 Predictors: Model reliance is concentrated in specific "V" (Vesta behavioral) and "C" (Count) features, which provide significantly more information than the raw transaction amount.
- Efficiency: Entire training process completed in 35.8 seconds with total features used - 217, making it highly scalable for large datasets.

**XGBoost - Top 15 Most Important Features**



XGBoost Model Statistics

- Trees: 500
- Max Depth: 8
- Learning Rate: 0.05
- Scale Pos Weight: 26.94
- Features Used: 217
- Top Feature: V70

# HGNN Model

- Graph-Based Fraud Architecture
- Concept: Moves beyond tabular analysis to Relationship Mapping.
- Heterogeneous Graph: Connects three distinct node types: Transactions, Cards, and Devices.
- Attention Mechanism: Uses a Transformer-based architecture to automatically weigh the importance of different connection types (e.g., identifying "High-Risk Devices").
- Temporal Intelligence: Incorporates Temporal Decay to prioritize recent behavioral patterns over historical data.
- Ideal Use Case: Best used as a high-confidence filter to automatically block transactions without needing manual review.

# Training Curves





# Preview

Deployed via Streamlit:  
<https://credit-card-fraud-detection-system-hyyu5xm7adgwjhhweyg8gt.streamlit.app/>

Upload CSV

CSV file

Upload

1GB per file • CSV

Select model(s)

decision\_tree x

xgboost x

hgnn x

Fraud threshold

0.50

Max rows to score (0 = all)

2000

Max columns to score (0 = all)

80

Supported input: full merged feature CSV, or id-only CSV (for example sample\_submission with TransactionID). Id-only files are auto-expanded to model features by TransactionID. Very large files (for example 0.7GB) can upload slowly on Streamlit Cloud; id-only CSV is usually much faster. Use row/column limits above

Fork

Fraud Detection Checker

Upload transaction data and run the three trained models: Decision Tree, XGBoost, and HGNN.

Decision Tree

Fast baseline

XGBoost

Best tabular model

HGNN

Graph-aware fraud model

Upload a CSV file to begin.

# Prediction Preview

Upload CSV

CSV file

train\_p...d\_50mb.csv

51.7MB

+

Select model(s)

decision\_tree

xgboost

hgnn

Fraud threshold

0.50

Max rows to score (0 = all)

2000

Max columns to score (0 = all)

0

Supported input: full merged feature CSV, or id-only CSV (for example sample\_submission with TransactionID). Id-only files are auto-expanded to model features by TransactionID. Very large files (for example 0.7GB) can upload slowly on Streamlit Cloud; id-only CSV is usually much faster. Use row/column limits above for faster test runs on large files. If `isFraud` is present, the app also shows accuracy metrics.

Prediction Summary

DT Fraud Flags

168

XGB Fraud Flags

26

HGNN Fraud Flags

35

Metrics vs Uploaded Labels

|   | Model         | ROC-AUC | Accuracy | Precision | Recall | F1     |
|---|---------------|---------|----------|-----------|--------|--------|
| 0 | Decision Tree | 0.7169  | 0.9105   | 0.0893    | 0.3659 | 0.1435 |
| 1 | XGBoost       | 0.8113  | 0.9755   | 0.3462    | 0.2195 | 0.2687 |
| 2 | HGNN          | 0.7301  | 0.966    | 0.1143    | 0.0976 | 0.1053 |

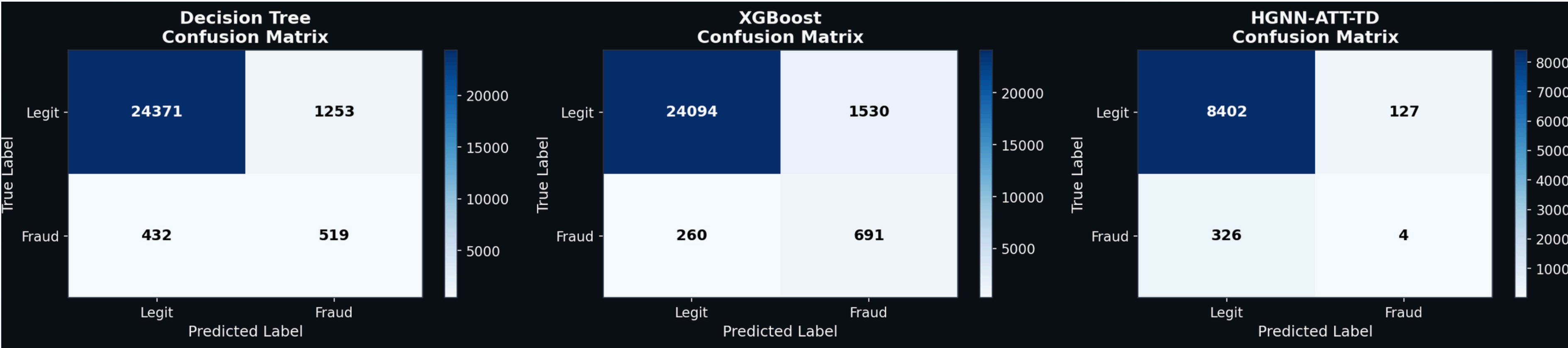
Per-Row Predictions

|    | TransactionID | decision_tree_probability | decision_tree_prediction | xgboost_probability | xgboost_prediction | hgnn_probability | hgnn_prediction | true_label | decision_tree_flag | xgboost_flag | hgnn_flag |
|----|---------------|---------------------------|--------------------------|---------------------|--------------------|------------------|-----------------|------------|--------------------|--------------|-----------|
| 0  | 2987000       | 0.1004                    | 0                        | 0.0575              | 0                  | 0.3209           | 0               | 0          | 0                  | 0            | 0         |
| 1  | 2987001       | 0.0579                    | 0                        | 0.0138              | 0                  | 0.3456           | 0               | 0          | 0                  | 0            | 0         |
| 2  | 2987002       | 0.022                     | 0                        | 0.0048              | 0                  | 0.3343           | 0               | 0          | 0                  | 0            | 0         |
| 3  | 2987003       | 0.2952                    | 0                        | 0.0026              | 0                  | 0.3268           | 0               | 0          | 0                  | 0            | 0         |
| 4  | 2987004       | 0.022                     | 0                        | 0.0206              | 0                  | 0.3525           | 0               | 0          | 0                  | 0            | 0         |
| 5  | 2987005       | 0.022                     | 0                        | 0.0063              | 0                  | 0.3433           | 0               | 0          | 0                  | 0            | 0         |
| 6  | 2987006       | 0.0707                    | 0                        | 0.0166              | 0                  | 0.3487           | 0               | 0          | 0                  | 0            | 0         |
| 7  | 2987007       | 0.3684                    | 0                        | 0.0629              | 0                  | 0.3695           | 0               | 0          | 0                  | 0            | 0         |
| 8  | 2987008       | 0.0579                    | 0                        | 0.0016              | 0                  | 0.3503           | 0               | 0          | 0                  | 0            | 0         |
| 9  | 2987009       | 0.0746                    | 0                        | 0.0014              | 0                  | 0.3081           | 0               | 0          | 0                  | 0            | 0         |
| 10 | 2987010       | 0.7568                    | 1                        | 0.0055              | 0                  | 0.3721           | 0               | 0          | 1                  | 0            | 0         |

Download predictions CSV

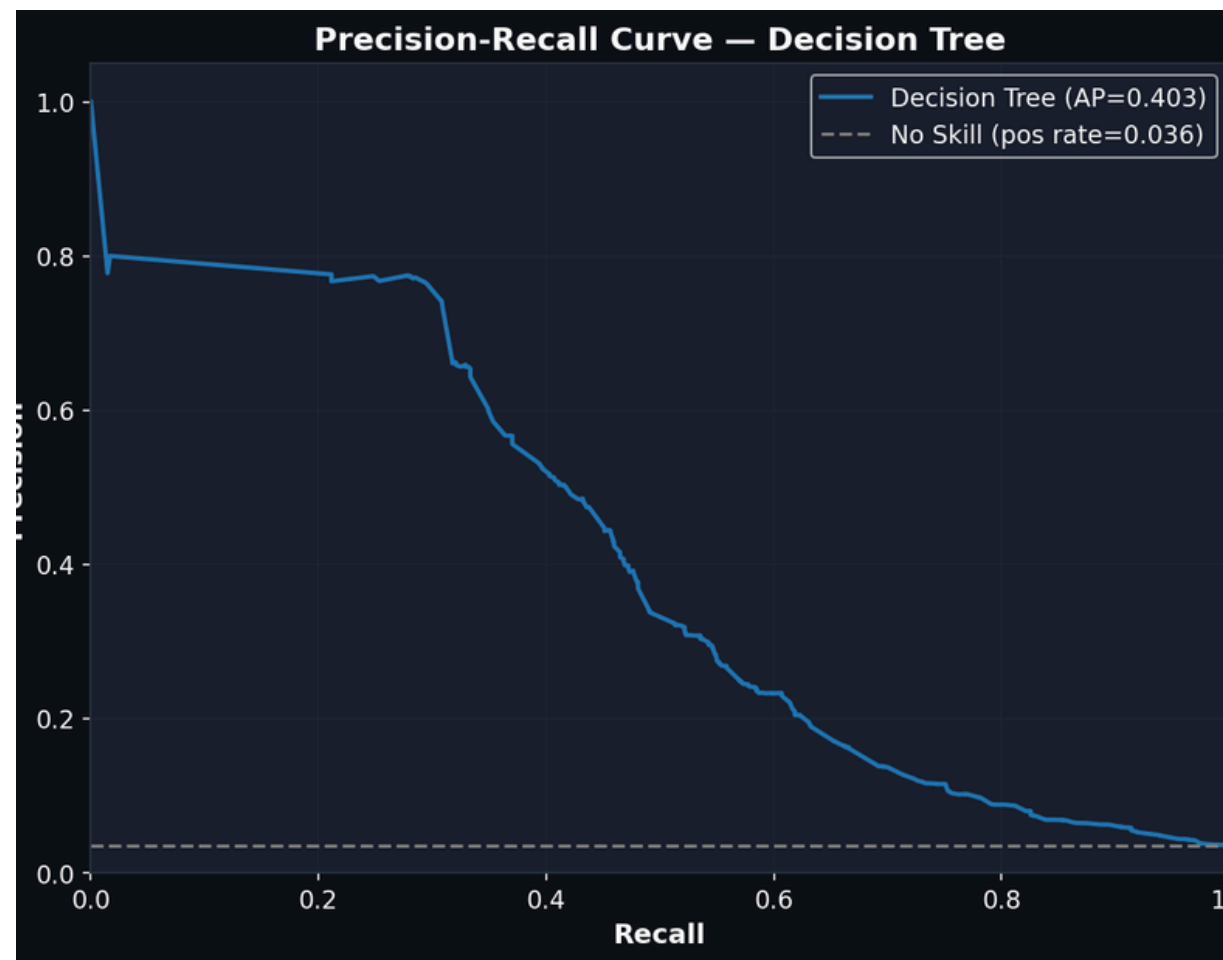
# Model Evaluation

## Confusion Matrices

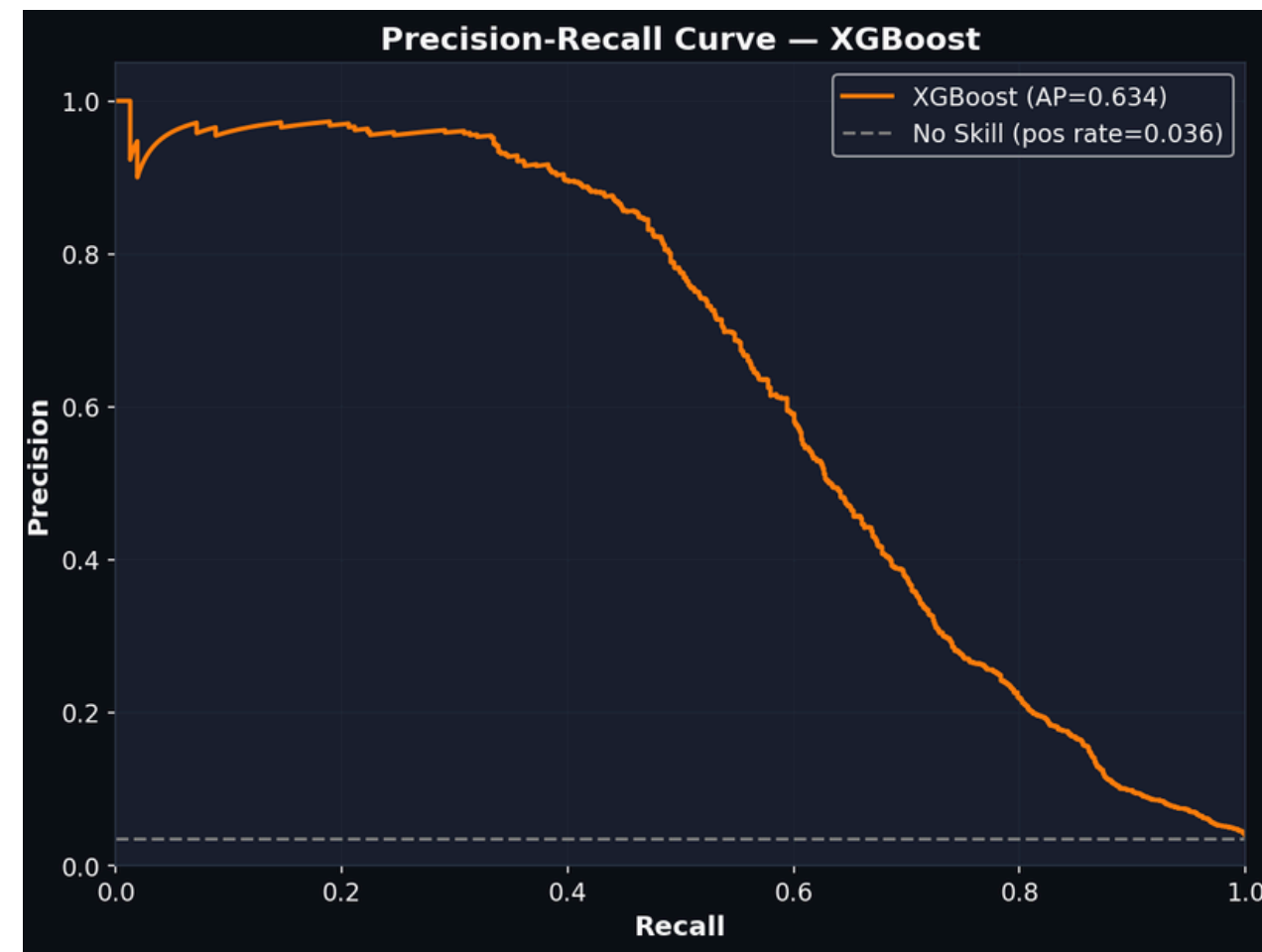


# Precision Recall Curves

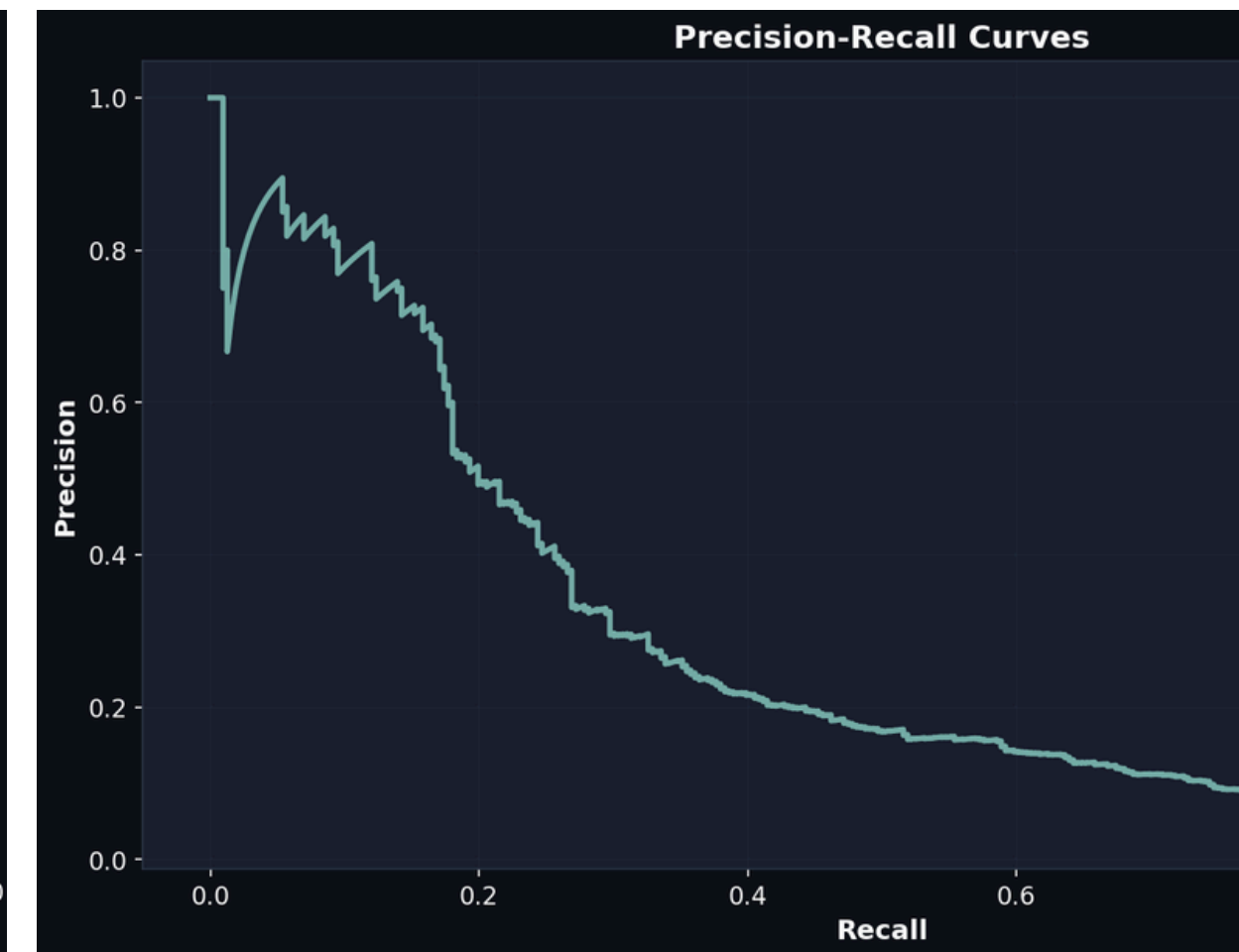
## Decision Tree



## XGBoost



## HGNN





| Metric                    | Decision Tree             | XGBoost          | HGNN-ATT-TD                |
|---------------------------|---------------------------|------------------|----------------------------|
| Accuracy                  | ~91%                      | 93.26%           | <b>96.42%</b>              |
| Recall (Fraud Catch Rate) | ~65%                      | <b>72.66%</b>    | 20.57%                     |
| Precision                 | ~28%                      | 31.11%           | <b>49.62%</b>              |
| False Positive Rate       | High (~8%)                | Moderate (5.97%) | <b>Ultra-Low (0.77%)</b>   |
| Training Speed            | <b>Fastest (&lt; 10s)</b> | Fast (35.8s)     | Slow (Minutes, GPU needed) |

# Conclusion

- HGNN-ATT-TD (Top Performer): Achieved highest accuracy (96.60%) with lowest false positives (0.77%); best for reliable, low-friction automated systems.
- Strong Detection: Correctly predicted 35/41 cases (~85%), making it the most effective overall model.
- XGBoost (Balanced Model): Highest ROC-AUC (0.8113); best at distinguishing fraud patterns and offers strong recall for deeper analysis.
- Decision Tree (High Sensitivity): Captures more fraud cases (higher recall) but with many false positives (over-flagging)
- For a production environment, the HGNN-ATT-TD is the superior choice for automated fraud prevention.
- By analyzing relationships (Graph data) rather than just independent rows (Tabular data), it provides the most stable and reliable detection with nearly zero impact on legitimate users.